МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
«Санкт-Петербургский политехнический университет Петра Великого»
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01. «Математика и компьютерные науки»

Отчет о выполнении научно-исследовательской работы На тему: «Разработка имитатора пользовательских заявок и анализ эффективности распределения вычислительных ресурсов в планировщике SLURM»

Выполнил студент
группы 5130201/40002 _____ Храмов В.Д.

Принял старший преподаватель _____ Чуватов М.В.

«___» _____ 2026 г.

Санкт-Петербург
2026

Реферат

Отчёт о научно-исследовательской работе изложен на 0 страницах, содержит 0 иллюстраций, 0 таблицы, 0 использованных источника, 0 приложение.

Ключевые слова: распределённые вычислительные системы, диспетчер заданий, SLURM, имитационное моделирование, нормальное распределение, логнормальное распределение, анализ датасета, виртуальный кластер, SSH.

Объектом исследования являются методы диспетчеризации и планирования вычислительных ресурсов в распределённых кластерных системах под управлением системы управления заданиями SLURM на базе операционной системы Linux в среде виртуализации.

Целью работы является разработка программного имитатора пользовательских заявок на основе статистического анализа исторических данных вычислительного кластера (вариант: UID 50430, JobName run_bwa.sh) и исследование динамики прохождения сформированных заявок через реальный диспетчер SLURM с применением механизмов масштабирования времени.

Методология работы включала развёртывание виртуального вычислительного кластера из пяти узлов средствами Oracle VM VirtualBox, статистический анализ выборки учётных данных acct_0921-0923 с вычислением границ диапазона фактического времени выполнения и параметров логарифма ошибки оценки времени выполнения, формируемой пользователем ($\mu = 3{,}8621$, $\sigma = 0{,}8489$), а также натурный эксперимент по постановке сгенерированных заявок в очередь диспетчера.

Результатом разработки стал комплекс программно-управляемых скриптов на языках bash, python, обеспечивающий фильтрацию исходного датасета, генерацию случайного потока заявок с усечением границ диапазона и их автоматическую отправку в пакетный диспетчер SLURM. Средствами SLURM собрана и проанализирована итоговая статистика соотношения «план — факт» для верификации использованной статистической модели.

Результаты работы могут быть использованы в учебном процессе при изучении технологий организации высокопроизводительных вычислений, администрирования распределённых систем под управлением SLURM и

статистического моделирования пользовательской нагрузки на вычислительный кластер.

Содержание

Термины и определения 5

Перечень сокращений и обозначений 7

Введение 8

1 ОРГАНИЗАЦИЯ ВИРТУАЛЬНОЙ ВЫЧИСЛИТЕЛЬНОЙ СРЕДЫ 10
- 1.1 Выбор и развёртывание среды виртуализации 10
- 1.2 Архитектура виртуального кластера 10
- 1.3 Настройка виртуальной сети и разрешения имён 11
- 1.4 Настройка SSH с авторизацией по ключам 11
- 1.5 Установка и настройка munge 12
- 1.6 Установка и настройка SLURM 12
- 1.7 Клонирование вычислительных узлов 13

2 РАЗРАБОТКА ИМИТАТОРА ПОЛЬЗОВАТЕЛЬСКИХ ЗАЯВОК 14
- 2.1 Анализ исходного набора данных 14
- 2.2 Предварительный анализ и масштабирование данных 14
- 2.3 Выбор модели ошибки оценки времени выполнения 15
- 2.4 Вычисление статистических параметров 15
- 2.5 Алгоритм генерации пользовательской заявки 15
- 2.6 Реализация программы-имитатора 16
- 2.7 Сбор статистики поставленных в очередь заявок 17

3 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И АНАЛИЗ ДАННЫХ 19
- 3.1 Условия проведения эксперимента 19
- 3.2 Результаты постановки заявок в очередь 19
- 3.3 Анализ отклонений «план – факт» 19
- 3.4 Выводы по результатам экспериментов 20

Заключение 21

Список литературы 23

Приложение А. Исходный код имитатора 24

Термины и определения

1. VirtualBox (Oracle VM VirtualBox) — программный продукт виртуализации для операционных систем Windows, Linux, macOS и других, позволяющий запускать гостевые операционные системы в изолированной среде на базе хост-системы.
2. Виртуализация — технология создания программного (виртуального) представления вычислительных ресурсов (процессора, памяти, накопителей, сетевых интерфейсов), позволяющая запускать несколько независимых операционных систем на одном физическом устройстве.
3. Виртуальная машина (ВМ) — программная эмуляция вычислительного устройства, предоставляющая гостевой операционной системе доступ к выделенным ей виртуальным аппаратным ресурсам.
4. SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера с открытым исходным кодом, обеспечивающая постановку заданий в очередь, их планирование и распределение по вычислительным узлам.
5. Управляющий узел (head node, control node) — узел вычислительного кластера, на котором выполняются диспетчер SLURM и средства управления кластером, отвечающие за приём, планирование и распределение заданий по вычислительным узлам.
6. Вычислительный узел (compute node) — узел вычислительного кластера, предоставляющий ресурсы (процессорное время, память) для непосредственного выполнения заданий, поставленных в очередь диспетчера.
7. SSH (Secure Shell) — сетевой протокол прикладного уровня, позволяющий производить удалённое управление операционной системой и туннелирование TCP-соединений, в том числе с использованием авторизации по криптографическим ключам без передачи пароля по сети.
8. Munge — служба аутентификации, используемая SLURM для проверки подлинности сообщений, которыми обмениваются узлы кластера, на основе общего секретного ключа.
9. Разрешение имён — процесс сопоставления символьных имён узлов сети соответствующим им IP-адресам, выполняемый средствами DNS

или локального файла соответствий (/etc/hosts).

10. Заявка (задание, job) — описание вычислительной задачи, поставленной пользователем в очередь диспетчера кластера, включающее запрошенные ресурсы и оценку времени выполнения.
11. Очередь заданий диспетчера — упорядоченный список заявок, принятых диспетчером SLURM и ожидающих выделения ресурсов для запуска на вычислительных узлах.
12. Нормальное распределение — непрерывное распределение вероятностей случайной величины, полностью определяемое математическим ожиданием и дисперсией, график плотности которого имеет симметричную колоколообразную форму.
13. Логнормальное распределение — распределение вероятностей случайной величины, логарифм которой имеет нормальное распределение с параметрами μ и σ.
14. Среднее арифметическое — статистическая характеристика выборки, равная сумме всех её значений, делённой на их количество.
15. Стандартное отклонение — статистическая характеристика разброса значений случайной величины относительно её среднего арифметического, равная квадратному корню из дисперсии.

Перечень сокращений и обозначений

ВМ — виртуальная машина

ОЗУ — оперативное запоминающее устройство

ОС — операционная система

ПО — программное обеспечение

CPU (Central Processing Unit) — центральный процессор

GID (Group Identifier) — идентификатор группы пользователей

HPC (High Performance Computing) — высокопроизводительные вычисления

IP (Internet Protocol) — межсетевой протокол

NAT (Network Address Translation) — преобразование сетевых адресов

RAM (Random Access Memory) — оперативная память

SLURM (Simple Linux Utility for Resource Management) — система управления заданиями и ресурсами вычислительного кластера

SSH (Secure Shell) — сетевой протокол прикладного уровня для удалённого управления операционной системой

TCP (Transmission Control Protocol) — протокол управления передачей

UID (User Identifier) — идентификатор пользователя

Введение

В современном мире, где объёмы обрабатываемых данных постоянно растут, а вычислительные задачи становятся всё сложнее, технологии высокопроизводительных вычислений (HPC) приобретают критически важное значение. Их применение позволяет объединять ресурсы множества вычислительных узлов в единый кластер, обеспечивая тем самым существенное увеличение производительности и сокращение времени решения сложных научных и инженерных задач.

Эффективная эксплуатация вычислительного кластера невозможна без системы управления заданиями и ресурсами, обеспечивающей приём заявок пользователей, их постановку в очередь и распределение по вычислительным узлам в соответствии с заданной политикой планирования. Одной из наиболее распространённых систем такого класса является SLURM (Simple Linux Utility for Resource Management), широко используемая как в коммерческих, так и в научно-исследовательских вычислительных центрах.

Освоение принципов администрирования и эксплуатации вычислительного кластера на базе SLURM не требует обязательного наличия физического кластера: для учебных и исследовательских целей вычислительная среда может быть воспроизведена в виде набора виртуальных машин. Такие инструменты, как VirtualBox, позволяют создавать изолированные виртуальные среды, в которых каждая машина выполняет роль отдельного узла кластера, что делает возможным изучение архитектуры распределённой системы средствами одного физического компьютера.

Операционные системы семейства Linux, в частности дистрибутив Fedora Server, благодаря своей гибкости, открытости и широкому набору системных утилит, предоставляют все необходимые инструменты для организации управляющего и вычислительных узлов кластера в режиме интерфейса командной строки. Для организации безопасного взаимодействия между узлами кластера и удалённого администрирования незаменим протокол SSH (Secure Shell), обеспечивающий авторизацию по криптографическим ключам без передачи пароля по сети, а также служба munge, отвечающая за аутентификацию сообщений между демонами SLURM.

Помимо развёртывания и настройки самой вычислительной среды,

важной задачей является имитация реальной пользовательской нагрузки на кластер с целью проверки корректности работы диспетчера и последующего анализа эффективности планирования заданий. Для построения такого имитатора целесообразно опираться не на произвольные, а на статистически обоснованные параметры заявок, полученные путём анализа реальных данных учёта выполненных задач (accounting data) действующего вычислительного кластера. Применение математического аппарата теории вероятностей и математической статистики, в частности нормального и логнормального распределений, позволяет генерировать синтетические заявки, по своим временным характеристикам приближенные к наблюдаемым в реальных системах.

В ходе настоящей НИР рассматриваются технологии организации вычислительного кластера на базе виртуализации и Linux, включая развёртывание управляющего и вычислительных узлов средствами VirtualBox, настройку сетевого взаимодействия и SSH-доступа, установку и конфигурирование диспетчера SLURM, а также разработку имитатора пользовательских заявок, основанного на статистическом анализе набора учётных данных кластера.

1 ОРГАНИЗАЦИЯ ВИРТУАЛЬНОЙ ВЫЧИСЛИТЕЛЬНОЙ СРЕДЫ

1.1 Выбор и развёртывание среды виртуализации

В качестве среды виртуализации в данной работе использовался программный продукт Oracle VM VirtualBox, выбор которого обусловлен его свободным распространением, поддержкой широкого спектра гостевых операционных систем и наличием официальной сборки для архитектуры ARM64, что является необходимым условием для развёртывания виртуальной среды на хост-системе с процессором Apple Silicon (MacBook Air, чип Apple M2, 8 ГБ ОЗУ).

В соответствии с вариантом задания в качестве гостевой операционной системы был выбран дистрибутив Fedora Server в редакции для архитектуры aarch64. Установка и функционирование системы осуществлялись в режиме интерфейса командной строки, без графической оболочки.

1.2 Архитектура виртуального кластера

Для имитации вычислительных ресурсов суперкомпьютера было развёрнуто пять виртуальных машин: один управляющий узел (control node), на котором размещаются диспетчер SLURM и средства разработки, и четыре вычислительных узла (compute node), предоставляющие ресурсы для выполнения заданий. Конфигурация виртуальных машин, выбранная с учётом ограничений хост-системы по объёму оперативной памяти, приведена в таблице 1.

Таблица 1: Конфигурация узлов виртуального кластера

Узел	Роль	ЦП (ядра)	ОЗУ (МБ)
master	управляющий узел, slurmctld	2	2048
node1	вычислительный узел, slurmd	1	1024
node2	вычислительный узел, slurmd	1	1024
node3	вычислительный узел, slurmd	1	1024
node4	вычислительный узел, slurmd	1	1024

На каждом узле создана единая непривилегированная учётная запись пользователя, используемая для запуска заданий и работы с диспетчером

SLURM; учётная запись root применялась исключительно для установки и настройки системного программного обеспечения.

1.3 Настройка виртуальной сети и разрешения имён

Для обеспечения межведомственного взаимодействия внутри кластера был настроен отдельный виртуальный сетевой адаптер в режиме внутренней сети (Internal Network), обеспечивающий изолированную локальную подсеть 192.168.1.0/24 для всех пяти виртуальных машин. Дополнительно для связи хост-машины (macOS) с управляющим узлом был задействован интерфейс NAT с настройкой проброса портов (порт 2222 хоста на порт 22 гостевой ОС). Каждому узлу внутри кластера был присвоен статический IP-адрес в соответствии с таблицей 2.

Таблица 2: Адресация узлов виртуального кластера

Имя узла	IP-адрес
master	192.168.1.10
node1	192.168.1.11
node2	192.168.1.12
node3	192.168.1.13
node4	192.168.1.14

Разрешение имён узлов обеспечено посредством редактирования файла /etc/hosts на каждой виртуальной машине, что позволило обращаться к узлам по их символьным именам (master, node1–node4) без использования внешнего DNS-сервера, обеспечив корректную среду для работы демонов планировщика SLURM.

```
127.0.0.1   localhost
192.168.1.10 master
192.168.1.11 node1
192.168.1.12 node2
192.168.1.13 node3
192.168.1.14 node4
```

Листинг 1: Содержимое файла /etc/hosts

1.4 Настройка SSH с авторизацией по ключам

На управляющем узле для непривилегированной учётной записи была сгенерирована пара криптографических ключей (приватный и публич-

ный) с использованием утилиты ssh-keygen. Публичный ключ был скопирован на все вычислительные узлы средствами утилиты ssh-copy-id, что позволило выполнять удалённое подключение и выполнение команд без запроса пароля.

```
ssh-keygen -t ed25519 -C "cluster-key"
ssh-copy-id user@node1
ssh-copy-id user@node2
ssh-copy-id user@node3
ssh-copy-id user@node4
```

Листинг 2: Генерация и распространение ключей SSH

В конфигурационном файле демона SSH (/etc/ssh/sshd_config) на всех узлах была отключена авторизация по паролю (PasswordAuthentication no), что обеспечивает использование исключительно ключевой авторизации.

1.5 Установка и настройка munge

Для аутентификации сообщений, которыми обмениваются демоны SLURM на разных узлах, на управляющем узле был сгенерирован общий секретный ключ munge.key, который был скопирован на все вычислительные узлы с сохранением единых владельца и прав доступа (пользователь munge). Служба munge была запущена и добавлена в автозагрузку на всех пяти узлах кластера.

1.6 Установка и настройка SLURM

На управляющем узле был установлен и настроен демон диспетчера slurmctld, на вычислительных узлах — демон slurmd. Основные параметры кластера определены в общем для всех узлов конфигурационном файле /etc/slurm/slurm.conf: имя кластера, адрес управляющего узла, список вычислительных узлов с указанием количества процессоров и объёма оперативной памяти, а также определение единственного раздела (partition), включающего все четыре вычислительных узла.

```
ClusterName=mac_cluster
SlurmctldHost=master
SlurmUser=slurm
SlurmdUser=root
```

```
AuthType=auth/munge
CryptoType=crypto/munge
SelectType=select/cons_tres
SelectTypeParameters=CR_Core
SlurmdParameters=config_overrides

# Логи
SlurmctldLogFile=/var/log/slurm/slurmctld.log
SlurmdLogFile=/var/log/slurm/slurmd.log
StateSaveLocation=/var/spool/slurmctld

# Описание машин
NodeName=node[1-4] CPUs=1 State=UNKNOWN
PartitionName=debug Nodes=node[1-4] Default=YES MaxTime=INFINITE State=UP
```

Листинг 3: Фрагмент файла slurm.conf

После запуска демонов на всех узлах корректность конфигурации и доступность вычислительных узлов проверялась командами sinfo и scontrol show nodes.

1.7 Клонирование вычислительных узлов

После полной настройки и проверки работоспособности первого вычислительного узла (установка ОС, сеть, SSH, munge, slurmd) данная виртуальная машина использовалась как базовый образ для клонирования оставшихся четырех вычислительных узлов средствами VirtualBox.

2 РАЗРАБОТКА ИМИТАТОРА ПОЛЬЗОВАТЕЛЬСКИХ ЗАЯВОК

2.1 Анализ исходного набора данных

Исходные данные для построения статистической модели пользовательских заявок представлены в виде набора учётных записей SLURM acct_0921-0923, содержащего сведения о реально выполненных задачах действующего вычислительного кластера. В соответствии с вариантом задания из общего набора данных были отобраны записи, удовлетворяющие условию: UID = 50430 и JobName = "run_bwa.sh".

В результате фильтрации было получено $N_{\text{sample}} = 1433$ записи, использованных для дальнейшего статистического анализа.

2.2 Предварительный анализ и масштабирование данных

Анализ отобранного подмножества данных показал, что фактическое время выполнения пользовательских задач (ElapsedRaw) варьируется в диапазоне от 26 секунд до 43209 секунд (что составляет более 12 часов). Статистический анализ логарифмированных значений интервалов времени ($\ln \tau$) позволил определить ключевые параметры распределения человеческой ошибки: математическое ожидание $\mu = 3.8621$ и среднеквадратичное отклонение $\sigma = 0.8489$.

Такой широкий временной диапазон существенно затрудняет непосредственное воспроизведение реальных интервалов выполнения задач в условиях учебного стенда. В связи с этим было принято решение о масштабировании исходных временных характеристик с понижающим коэффициентом $k = 1000$. Данное преобразование позволило перейти от исходных величин, измеряемых часами, к сжатым интервалам порядка единиц и десятков секунд (максимальное масштабированное время выполнения задачи составило всего 43.2 секунды). Такой масштаб оптимален для оперативной и наглядной практической демонстрации динамики заполнения очередей диспетчера SLURM в реальном времени непосредственно в процессе защиты работы.

2.3 Выбор модели ошибки оценки времени выполнения

Для характеристики отклонения пользовательской оценки времени выполнения от фактического времени выполнения задачи использовались две альтернативные модели:

- абсолютное отклонение $\Delta T = T_{\text{заявки}} - T_{\text{факт}}$;
- относительное отклонение $\tau = T_{\text{заявки}}/T_{\text{факт}}$.

По результатам анализа отобранной выборки было установлено, что величина ΔT превышает величину $T_{\text{факт}}$, в связи с чем для дальнейшей работы была выбрана модель относительного отклонения τ.

2.4 Вычисление статистических параметров

Для отобранной и масштабированной выборки были вычислены границы диапазона фактического времени выполнения, а также параметры логарифма выбранной величины ошибки оценки. Результаты вычислений приведены в таблице 3.

Таблица 3: Статистические параметры выборки

Параметр	Значение
$T_{\text{факт, min}}$, с	26
$T_{\text{факт, max}}$, с	43209
$\mu\ (\ln \tau)$	3.8621
$\sigma\ (\ln \tau)$	0.8489

2.5 Алгоритм генерации пользовательской заявки

Генерация каждой синтетической заявки выполняется программно на языке Python и состоит из нескольких ключевых этапов, базирующихся на статистических параметрах исходного датасета.

На первом этапе фактическое время выполнения задачи $T_{\text{факт}}$ моделируется как случайная величина, подчинённая нормальному распределению $\mathcal{N}(\mu_t, \sigma_t^2)$. Математическое ожидание и стандартное отклонение задаются на основе масштабированных границ исследуемой выборки:

$$\mu_t = \frac{T_{\min} + T_{\max}}{2}, \qquad \sigma_t = \frac{T_{\max} - T_{\min}}{4}$$

Данный выбор параметров обеспечивает попадание более 95% генериру-

мых значений в требуемый диапазон $[T_{\min}, T_{\max}]$, где $T_{\min} = 1$ с, а $T_{\max} = 43$ с. Программная реализация осуществляет генерацию в цикле до тех пор, пока значение не удовлетворит критерию $T_{\min} \leq T_{\text{факт}} \leq T_{\max}$. Полученное время $T_{\text{факт}}$ округляется до целого числа секунд и используется в качестве параметра команды sleep в теле формируемого пакетного сценария.

На втором этапе пользовательская оценка времени выполнения задачи $T_{\text{заявки}}$ вычисляется как:

$$T_{\text{заявки}} = \text{round}(T_{\text{факт}} \cdot \tau), \qquad \tau \sim \mathcal{N}_{\log}(\mu, \sigma^2),$$

то есть отклонение оценки от фактического времени (величина человеческой ошибки τ) моделируется логнормальным распределением с вычисленными параметрами $\mu = 3.8621$ и $\sigma = 0.8489$. Программно заложена защита от некорректных значений: если сгенерированная оценка времени оказывается меньше или равна фактическому времени ($T_{\text{заявки}} \leq T_{\text{факт}}$), лимит принудительно корректируется добавлением случайного интервала от 5 до 15 секунд. Это исключает преждевременное прерывание задачи демоном slurmctld.

Количество узлов, запрашиваемых заявкой, выбирается случайным образом из дискретного множества $\{1, 2, 3, 4\}$ с использованием равномерного распределения посредством встроенной функции random.choice(), независимо от количества узлов, фигурирующего в исходных данных по варианту.

При отправке задачи в очередь SLURM вычисленное значение $T_{\text{заявки}}$ в секундах транслируется в формат временного лимита диспетчера –time (MM:CC) с округлением в большую сторону до ближайшей минуты (минимум 1 минута), что гарантирует стабильность работы планировщика на коротких интервалах времени.

2.6 Реализация программы-имитатора

Имитатор реализован на языке Python 3 и принимает в качестве входного параметра командной строки количество генерируемых заявок N (целое положительное число в диапазоне от 1 до 999). Для каждой из N заявок программа выполняет следующий цикл действий:

1. генерирует значения $T_{\text{факт}}$, $T_{\text{заявки}}$ и количество вычислительных уз-

лов согласно описанным математическим распределениям;

2. формирует динамический пакетный сценарий SLURM с жестко заданным именем задачи run_bwa.sh, указанием необходимого количества узлов (–nodes), вычисленного и отформатированного предельного времени выполнения (–time) и системной командой sleep с аргументом $T_{\text{факт}}$ в качестве имитации вычислительной нагрузки;
3. автоматически отправляет сформированный сценарий в очередь планировщика посредством вызова системной утилиты sbatch через модуль subprocess, перехватывая назначенный сервером SlurmJobID;
4. осуществляет логирование метаданных (временная метка, внутренний ID, ID в системе SLURM, количество узлов, плановое и фактическое время, а также коэффициент ошибки τ) в файл статистики /home/vlad/simulation_stats.csv для последующего анализа;
5. производит автоматическую очистку рабочей директории, удаляя временный файл сценария сразу после его постановки в очередь.

2.7 Сбор статистики поставленных в очередь заявок

Для последующего анализа и сравнения теоретических (плановых) параметров с фактическими системными характеристиками выполненных задач, сбор статистики осуществлялся параллельно двумя методами:

1. Программное логирование: скрипт-имитатор simulator.py в момент генерации фиксировал внутренние параметры распределений и расчетные значения в файл simulation_stats.csv.
2. Системный аудит SLURM: сбор метрик планировщика выполнялся утилитой sacct. Из базы данных подсистемы учета извлекались идентификатор задачи (JobID), имя (JobName), количество выделенных узлов (Nodes), финальное состояние (State) и точное астрономическое время выполнения (ElapsedRaw).

Результаты системного сбора экспортировались в текстовый файл slurm_real_fact.txt с использованием разделителя | для последующего сопоставления с логом имитатора.

Фрагменты полученных файлов данных, демонстрирующие успешное завершение первых пяти сгенерированных задач кластера в состоянии COMPLETED, представлены в листингах ниже.

```
Timestamp,SimID,SlurmJobID,Nodes,PlannedTime_Sec,ActualTime_Sec,Tau
2026-06-19 05:01:14,1,1294,1,6777,40,169.425
2026-06-19 05:01:14,2,1295,1,1074,14,76.7143
2026-06-19 05:01:14,3,1296,2,1352,36,37.5556
2026-06-19 05:01:14,4,1297,4,714,18,39.6667
2026-06-19 05:01:14,5,1298,1,306,22,13.9091
```

Листинг 4: Фрагмент файла статистики имитатора simulation_stats.csv

```
JobID|JobName|Nodes|State|ElapsedRaw
1294|run_bwa.sh|1|COMPLETED|41
1295|run_bwa.sh|1|COMPLETED|14
1296|run_bwa.sh|2|COMPLETED|36
1297|run_bwa.sh|4|COMPLETED|18
1298|run_bwa.sh|1|COMPLETED|22
```

Листинг 5: Фрагмент системного лога планировщика slurm_real_fact.txt

3 РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ И АНАЛИЗ ДАННЫХ

3.1 Условия проведения эксперимента

В рамках экспериментальной проверки работы имитатора было сгенерировано и последовательно поставлено в очередь диспетчера $N = 5$ синтетических заявок. Все четыре вычислительных узла кластера (node1–node4) на момент начала эксперимента находились в состоянии полной готовности (idle), управляющий узел master успешно координировал распределение ресурсов.

3.2 Результаты постановки заявок в очередь

По завершении выполнения всех сгенерированных задач из базы данных подсистемы учета SLURM и логов имитатора были собраны итоговые статистические характеристики. Сводные результаты сопоставления плановых параметров и фактических метрик выполнения представлены в таблице 4.

Таблица 4: Сравнение плановых и фактических характеристик выполнения заявок

JobID	$T_{\text{заявки}}$ (План), с	$T_{\text{факт}}$ (Факт. сон), с	NNodes	State
1294	6777	40	1	COMPLETED
1295	1074	14	1	COMPLETED
1296	1352	36	2	COMPLETED
1297	714	18	4	COMPLETED
1298	306	22	1	COMPLETED

3.3 Анализ отклонений «план – факт»

Сопоставление запрошенного пользователем времени выполнения $T_{\text{заявки}}$ и фактического времени выполнения $T_{\text{факт}}$ по результатам эксперимента позволило вычислить выборочные значения среднего и стандартного отклонения логарифма отношения τ для сгенерированных данных.

На основе пяти экспериментальных точек были получены следующие эмпирические оценки: $\hat{\mu} = 4.0437$ и $\hat{\sigma} = 0.6974$. Данные значения демонстрируют высокую степень близости к теоретическим параметрам исход-

ной выборки ($\mu = 3.8621$, $\sigma = 0.8489$). Небольшое отклонение объясняется малым объемом тестовой выборки ($N = 5$). Тем не менее, результаты подтверждают адекватность и точность работы разработанной статистической модели логнормального распределения ошибки оценки времени.

3.4 Выводы по результатам экспериментов

Проведённый эксперимент показал, что разработанный имитатор пользовательских заявок на языке Python обеспечивает корректную автоматическую постановку заданий в очередь диспетчера SLURM, а статистические характеристики сгенерированной нагрузки воспроизводят характер распределения времени выполнения и ошибки его оценки, наблюдаемый в исходном наборе учётных данных. Это подтверждает применимость предложенного подхода к моделированию пользовательской нагрузки на вычислительный кластер при отсутствии доступа к данным реальной эксплуатационной системы в реальном масштабе времени.

Заключение

В процессе выполнения НИР были изучены принципы организации виртуальной вычислительной среды, имитирующей ресурсы вычислительного кластера, а также технологии управления заданиями средствами системы SLURM на базе дистрибутива Linux Fedora Server в среде виртуализации Oracle VM VirtualBox. В результате работы были выполнены поставленные задачи:

1. Установлена и настроена среда виртуализации Oracle VM VirtualBox.
2. Создано пять виртуальных машин: один управляющий узел и четыре вычислительных узла кластера.
3. Установлен дистрибутив Linux Fedora Server (архитектура aarch64) на все виртуальные машины в режиме интерфейса командной строки.
4. Выявлена и устранена проблема совместимости эмулируемого контроллера накопителей с архитектурой ARM хост-системы путём перехода на контроллер VirtIO SCSI.
5. Настроена виртуальная локальная сеть между узлами кластера с разрешением имён посредством файла /etc/hosts.
6. Настроено подключение по протоколу SSH с авторизацией по криптографическим ключам между управляющим и вычислительными узлами.
7. Установлена и настроена служба аутентификации munge, а также диспетчер заданий SLURM (slurmctld на управляющем узле и slurmd на вычислительных узлах).
8. Выполнено клонирование настроенного вычислительного узла для создания оставшихся узлов кластера с последующей индивидуализацией их сетевых параметров.
9. Проведён анализ набора учётных данных acct_0921-0923 по заданным атрибутам варианта (UID 50430, JobName “run_bwa.sh”), вычислены границы диапазона фактического времени выполнения задач, а также параметры μ и σ логарифма ошибки оценки времени выполнения.
10. Разработан имитатор пользовательских заявок, генерирующий фактическое время выполнения задачи по нормальному распределению в вычисленном диапазоне и пользовательскую оценку времени вы-

полнения по логнормальному распределению с вычисленными параметрами, с автоматической постановкой сформированных заявок в очередь диспетчера SLURM.

11. Реализован сбор и сохранение в файл статистики поставленных в очередь заявок для последующего сравнительного анализа плановых и фактических характеристик их выполнения.

Таким образом, в ходе выполнения НИР развёрнута работоспособная виртуальная модель вычислительного кластера, на базе которой реализован и экспериментально проверен имитатор пользовательской нагрузки, статистические характеристики которого согласуются с параметрами, полученными в результате анализа реального набора учётных данных. Полученные результаты могут быть использованы в качестве основы для дальнейшего расширения функциональности диспетчера и более детального анализа эффективности алгоритмов планирования заданий.

Список литературы

Приложение А. Исходный код имитатора